Freely-Walking Optomotor Experiments

Laura Burnett – Reiser Lab, HHMI Janelia Research Campus

2026-02-23

Table of contents

1 Quick Reference **4**
- 1.1 Pipeline at a Glance . . . 4
- 1.2 Key Files . . . 4
- 1.3 Behavioral Metrics Quick Reference . . . 4
- 1.4 Common MATLAB Access Patterns . . . 5
- 1.5 Glossary of Key Terms . . . 5

2 Introduction and Overview **7**
- 2.1 Overview of the Freely-Walking Optomotor Experiments . . . 7
- 2.2 The Full Data Pipeline . . . 7

3 Automated Data Pipeline **10**
- 3.1 Overview . . . 10
- 3.2 Stage 1: Monitoring and Copying . . . 12
  - 3.2.1 `monitor_and_copy.py` . . . 12
- 3.3 Stage 2: Automated Tracking . . . 13
  - 3.3.1 `monitor_and_track.py` . . . 13
  - 3.3.2 `batch_track_ufmf.m` . . . 14
- 3.4 Stage 3: Daily Processing . . . 14
  - 3.4.1 `daily_processing.py` . . . 14
- 3.5 Supporting Scripts . . . 15
  - 3.5.1 `reprocessing_script.py` . . . 15
  - 3.5.2 `copy_movies_to_network.py` . . . 15
- 3.6 Configuration and Deployment . . . 16

4 Running an Experiment – Protocol Functions **17**
- 4.1 Overview of Protocol Structure . . . 17
  - 4.1.1 Typical Experiment Flow (protocol_27.m) . . . 18
- 4.2 Initialization Functions . . . 18
  - 4.2.1 `get_input_parameters.m` . . . 18
  - 4.2.2 `initialize_video_and_folders.m` . . . 18
  - 4.2.3 `initialize_temp_recording.m` . . . 19
  - 4.2.4 `get_temp_rec.m` . . . 19
- 4.3 Stimulus Presentation Functions . . . 19
  - 4.3.1 The `all_conditions` Matrix . . . 20
  - 4.3.2 Variant Comparison . . . 20
  - 4.3.3 `present_optomotor_stimulus.m` – Standard Bidirectional . . . 21
  - 4.3.4 `present_optomotor_stimulus_w_interval.m` – Per-Trial Intervals . . . 21
  - 4.3.5 `present_optomotor_stimulus_one_direction.m` – Single Direction . . . 22
  - 4.3.6 `present_optomotor_stimulus_curtain.m` – Curtain Patterns . . . 22
  - 4.3.7 `present_optomotor_stimulus_curtain_w_interval.m` . . . 22
  - 4.3.8 `present_optomotor_stimulus_diff_contrasts.m` – Contrast Variation . . . 22
  - 4.3.9 `present_fixation_stimulus.m` – Static Bar Display . . . 22

4.3.10 `present_shifted_stimulus.m` – Open-Loop Shifted Patterns 23
4.3.11 `present_acclim_off.m` – Dark Acclimation 23
4.4 Export Function 23
4.4.1 `export_to_google_sheets.m` 23

5 The LOG Data Structure 24
5.1 How the LOG Is Built 24
5.2 LOG.meta Fields 26
5.3 LOG.acclim_off1 and LOG.acclim_off2 Fields 26
5.4 LOG.acclim_patt Fields 27
5.5 LOG.log_N Fields (Stimulus Conditions) 27
5.6 Frame Synchronization 28

6 Data Processing Pipeline 29
6.1 `process_freely_walking_data.m` – The Orchestrator 29
6.2 `process_data_features.m` – Single Experiment Processing 30
6.3 `combine_data_one_cohort.m` – Core Feature Extraction 31
6.3.1 Step 1: Remove Bad Tracking (`check_tracking_FlyTrk`) 31
6.3.2 Step 2: High-Velocity Filtering 31
6.3.3 Step 3: Interpolation 31
6.3.4 Step 4: Distance from Center 32
6.3.5 Step 5: Angular Velocity (`vel_estimate`) 32
6.3.6 Step 6: Forward Velocity (`gaussian_conv`) 32
6.3.7 Step 7: Three-Point Velocity (`calculate_three_point_velocity`) 32
6.3.8 Step 8: Turning Rate (Curvature) 33
6.3.9 Step 9: Viewing Distance (`calculate_viewing_distance`) 33
6.3.10 Step 10: Inter-Fly Distance and Angle (`calculate_distance_to_nearest_fly`) 34
6.3.11 Output: `comb_data` Struct 34
6.4 `comb_data_one_cohort_cond.m` – Splitting by Condition 34
6.5 `comb_data_across_cohorts_cond.m` – Merging Across Experiments 35

7 The DATA Struct 35
7.1 Hierarchy 35
7.2 Condition Names 37
7.3 Per-Condition Fields 37
7.4 Accessing Data – Examples 37

8 Behavioral Metrics Reference 39
8.1 Forward Velocity (`fv_data`) 39
8.2 Angular Velocity (`av_data`) 40
8.3 Turning Rate (`curv_data`) 40
8.4 Distance from Center (`dist_data`) 40
8.5 Three-Point Velocity (`vel_data`) 40
8.6 Viewing Distance (`view_dist`) 41
8.7 Inter-Fly Distance (`IFD_data`) 41

8.8 Inter-Fly Angle (`IFA_data`) . . . 41
8.9 Position and Heading . . . 41

9 Visualization and Plotting Functions **42**
9.1 `make_overview.m` . . . 42
9.2 `plot_all_features_filt.m` . . . 42
9.3 `plot_all_features_acclim.m` . . . 42
9.4 `plot_allcond_onecohort_tuning.m` . . . 42

10 Utility Functions **43**
10.1 `check_strain_typos.m` . . . 43
10.2 `generate_circ_stim_ufmf.m` . . . 43

11 Potential Pitfalls **44**
11.1 Hardcoded Arena Parameters . . . 44
11.2 Hardcoded Project Paths . . . 44
11.3 Protocol Condition Arrays Are Per-Protocol . . . 44
11.4 Exactly 2 Repetitions Assumed . . . 44
11.5 Velocity Threshold of 50 mm/s . . . 45
11.6 300-Frame Pre-Buffer Is Hardcoded . . . 45
11.7 LOG Field Naming Is Fragile . . . 45
11.8 Pattern-to-Description Mapping . . . 45
11.9 Tracking QC Uses Statistical Mode . . . 46
11.10 Strain Typos Pass Silently . . . 46
11.11 Move-Folder Nesting Bug in Daily Processing . . . 46
11.12 No Retry Logic for Network File Operations . . . 46
11.13 Duplicated Path Constants Across Automation Scripts . . . 47

A Function Index **48**

1 Quick Reference

Outlined below are a few of the key concepts, files, and data structures that are essential for understanding and working with the freely-walking optomotor experiment pipeline. This section serves as a high-level overview and quick reference guide before diving into the detailed explanations in the subsequent sections.

1.1 Pipeline at a Glance

1. **Run the experiment** – `protocol_27.m` controls the LED arena and camera, producing a **UFMF video** and a **LOG.mat** file with stimulus timing.
2. **Track the flies** – `FlyTracker` processes the video offline, producing `feat.mat` (features) and `trx.mat` (trajectories).
3. **Compute behavioral metrics** – `combine_data_one_cohort(feat, trx)` filters bad tracking, interpolates gaps, and computes 12 behavioral metrics.
4. **Split by condition** – `comb_data_one_cohort_cond(LOG, comb_data)` uses LOG frame indices to slice continuous data into per-condition segments.
5. **Merge experiments** – `comb_data_across_cohorts_cond(protocol_dir)` combines all sessions into a single hierarchical DATA struct.

Steps 2–3 can be fully automated – see Section 3.

1.2 Key Files

File	Purpose
`protocols/protocol_27.m`	Example protocol script (defines conditions, runs experiment)
`protocols/protocol_functions/`	14 shared functions for initialization, stimulus, acclimation
`processing_functions/process_freely_walking_data.m`	Top-level processing entry point (by date)
`processing_functions/process_data_features.m`	Single-experiment processing orchestrator
`processing_functions/functions/combine_data_one_cohort.m`	Core: computes 12 behavioral metrics from tracking data
`processing_functions/functions/comb_data_one_cohort_cond.m`	Splits continuous data by condition using LOG frame indices
`processing_functions/functions/comb_data_across_cohorts_cond.m`	Merges experiments into final DATA struct
`monitor_and_copy_folder/monitor_and_copy.py`	Automated copy from acquisition PC to network
`monitor_and_track_folder/monitor_and_track.py`	Automated FlyTracker batch tracking
`daily_processing/daily_processing.py`	Automated daily processing and result distribution
`tracking/batch_track_ufmf.m`	Batch FlyTracker invocation for one date folder

1.3 Behavioral Metrics Quick Reference

Metric	Field	Units	Typical Range	Description
Forward velocity	`fv_data`	mm/s	0–30	Speed along heading direction
Angular velocity	`av_data`	deg/s	-500 to +500	Heading change rate (turning speed)
Turning rate	`curv_data`	deg/mm	Variable	Path curvature (av / fv)
Distance from center	`dist_data`	mm	0–120	Distance from arena center
Three-point velocity	`vel_data`	mm/s	0–50	Overall speed (any direction)
Viewing distance	`view_dist`	mm	0–240	Distance to wall along heading
Inter-fly distance	`IFD_data`	mm	0–240	Distance to nearest fly
Inter-fly angle	`IFA_data`	deg	-180 to 180	Angle to nearest fly vs heading
X position	`x_data`	mm	-120 to 120	Horizontal position
Y position	`y_data`	mm	-120 to 120	Vertical position
Heading (unwrapped)	`heading_data`	deg	Continuous	Cumulative heading angle
Heading (wrapped)	`heading_wrap`	deg	-180 to 180	Heading within one revolution

1.4 Common MATLAB Access Patterns

```
% Load combined data
DATA = comb_data_across_cohorts_cond( ...
    '/path/to/results/protocol_27');

% Access forward velocity:
%   strain, sex F, cohort 1, Rep 1 condition 1
fv = DATA.jfrc100_es_shibire_kir ...
    .F(1).R1_condition_1.fv_data;
%   fv is [n_flies x n_frames]

% Mean across flies at each timepoint
mean_fv = mean(fv, 1, 'omitnan');

% Number of cohorts for a strain/sex
n_cohorts = length( ...
    DATA.jfrc100_es_shibire_kir.F);

% List all strains
strain_names = fieldnames(DATA);
```

1.5 Glossary of Key Terms

Term	Definition
LOG	MATLAB struct saved during experiment; contains stimulus timing, metadata, and frame indices
comb_data	Intermediate struct with 12 behavioral metric matrices for one experiment
DATA	Final hierarchical struct organizing all experiments by strain, sex, cohort, and condition
FlyTracker	Offline tracking software that extracts fly positions and features from video
UFMF	Micro Fly Movie Format – compressed video format used by the overhead camera
Panel_com	MATLAB function that sends commands to the G3 LED arena controller
BIAS	Camera control software (Basic Image Acquisition Software)
G3 arena	Cylindrical LED display (72 panels, 3 rows x 192 columns)
PPM	Pixels per millimeter (calibration constant: 4.1691)
Condition	One stimulus configuration (pattern + speed + duration)
Repetition	One complete pass through all conditions (each experiment has 2 reps)
Cohort	One experimental session (one group of flies)
watchdog	Python library for filesystem event monitoring; used by `monitor_and_copy.py`
Task Scheduler	Windows service that runs batch files on a schedule; launches all automation scripts
0_unprocessed	Network directory for raw experiment data awaiting tracking
1_tracked	Network directory for tracked data awaiting MATLAB processing
2_processed	Network directory for fully processed data ready for analysis

2 Introduction and Overview

At a Glance

- Flies walk freely within a cylindrical arena while visual patterns are displayed on LED panels around the circumference facing inwards.
- An overhead camera records behavior at 30 fps in near infra-red light and saves the data in the compressed UFMF format.
- A MATLAB protocol script controls the arena and logs stimulus timing and parameters.
- The primary response measured is the **optomotor response** (corrective turning in the direction of motion).
- Data flows through 5 stages: experiment, tracking, metrics, condition-splitting, merging across cohorts of the same genetic strain.

2.1 Overview of the Freely-Walking Optomotor Experiments

These experiments study the visual-motor behaviour of freely-walking *Drosophila melanogaster* (fruit flies) inside a cylindrical LED arena (3rd generation - G3 arena). Groups of between 10-20 flies (ideally 15) walk freely on a flat surface of the underlying LED backlight while visual patterns – such as moving gratings, curtains, or stationary bars – are displayed on the surrounding LED panels. An overhead camera records the flies' behavior at 30 frames per second.

The primary behavioural response measured is the **optomotor response**: when a grating pattern moves across the visual field, walking flies tend to turn in the direction of motion. By varying stimulus parameters (pattern type, speed, contrast, spatial frequency), we can probe the neural circuits underlying motion detection. The experiments also include control conditions (e.g., static patterns, dark intervals) and a pre/post-stimulus acclimation period in darkness to measure baseline behavior.

In the summer of 2024, while performing the first round of these experiments, we observed another behaviour that we call the **centring response**: while the grating stimuli are being presented and flies perform the well-characterised optomotor turning, they also tend to walk towards the center of the arena. The response was colloquially given the moniker the *"hokey-cokey"* after its resemblance to the centripetal movement within the group dance performed by several cultures around the world. Many of the protocols designed for this arena were designed to try and. investigate this centring behaviour, it's reliance on stimulus parameters and its interaction with the optomotor response.

Arena geometry:

- 72 LED panels arranged in a cylinder (3 rows of 24 panels)
- Each panel is 8 x 8 pixels, giving 3 rows x 192 columns total
- Arena radius: approximately 120 mm
- Video recording at 30 fps in UFMF (compressed) format
- The LEDs emit light with a peak in the green range of the visible spectrum (~500nm).

2.2 The Full Data Pipeline

The data flows through several stages from experiment to analysis:

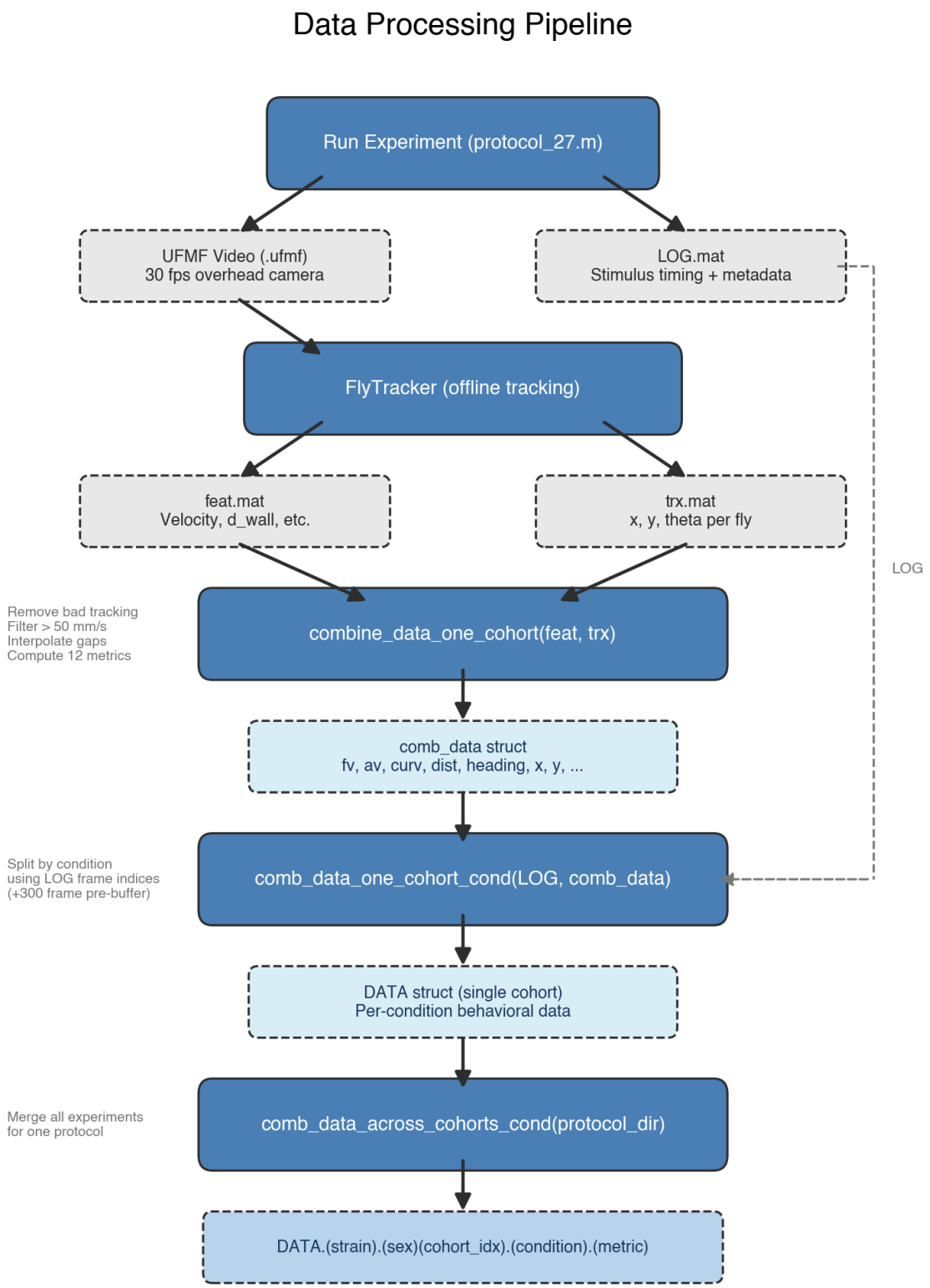

Figure 1: Data processing pipeline: from running the experiment to the final DATA struct.

1. **Run the experiment** – A MATLAB protocol script controls the LED arena and camera, producing a UFMF video file and a LOG.mat file with stimulus timing.
2. **Track the flies** – FlyTracker (offline software) processes the video to extract each fly's position, heading, and basic features, producing `feat.mat` and `trx.mat`.
3. **Compute behavioral metrics** – `combine_data_one_cohort` filters bad tracking, interpolates gaps, and computes 12 behavioral metrics (forward velocity, angular velocity, turning rate, etc.).
4. **Split by condition** – `comb_data_one_cohort_cond` uses the LOG frame indices to separate the continuous data into per-condition segments.
5. **Merge experiments** – `comb_data_across_cohorts_cond` combines data from multiple experimental sessions into a single hierarchical DATA struct for group analysis.

Steps **2** and **3** are automated by the pipeline scripts described in Section 3. When running, the automation handles copying, tracking, and processing without manual intervention.

3 Automated Data Pipeline

At a Glance

- Three Python scripts (written by **Aparna Dev**) automate data flow from the acquisition PC to processed results
- **Stage 1:** `monitor_and_copy.py` watches the acquisition PC and copies completed experiments to the network
- **Stage 2:** `monitor_and_track.py` polls the network, runs FlyTracker batch tracking, and moves data to `1_tracked`
- **Stage 3:** `daily_processing.py` runs `process_freely_walking_data` on tracked data and distributes results to the network
- The scripts run on two different physical PCs and communicate via shared network directories

3.1 Overview

The automated data pipeline eliminates manual file management between running an experiment and analysing the results. Three Python scripts, launched via Windows Task Scheduler batch files, handle the progression of data through a series of network directories. Each script monitors for new data, processes it, and advances it to the next stage.

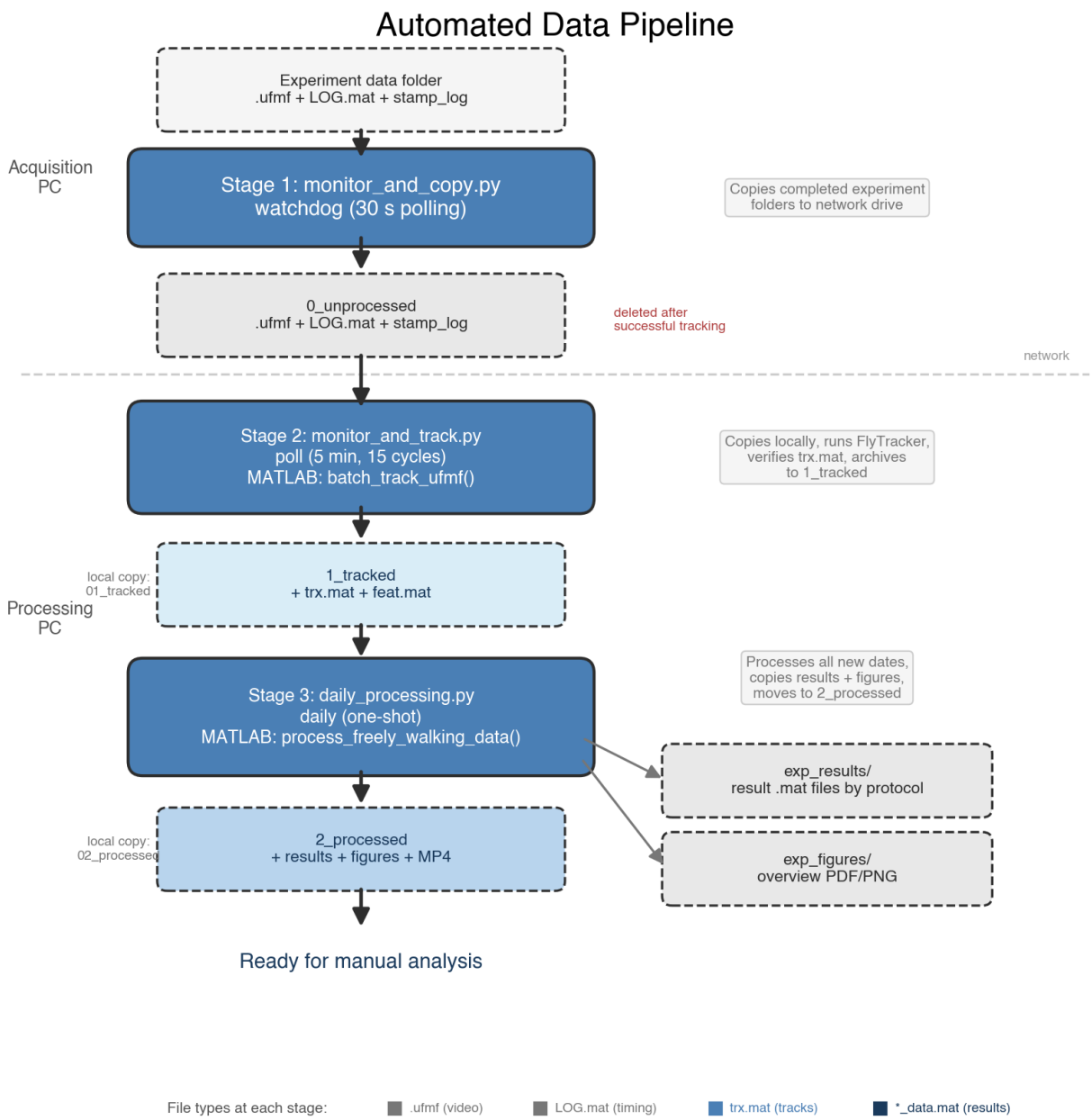

Figure 2: Automated data pipeline: three Python scripts move data through network directory stages from acquisition to processed results.

Three-stage summary:

Stage	Script	Runs On	Input	Output	Polling
1. Copy	`monitor_and_copy.py`	Acquisition PC	Local experiment data	Network `0_unprocessed`	30 s (watchdog)
2. Track	`monitor_and_track.py`	Processing PC	Network `0_unprocessed`	Network `1_tracked`	5 min, 15 cycles
3. Process	`daily_processing.py`	Processing PC	Local `01_tracked`	Network `2_processed`	One-shot (daily)

Network directory progression:

Directory	Contents	State
`0_unprocessed`	Raw experiment folders (.ufmf + LOG.mat + stamp_log)	Awaiting tracking
`1_tracked`	Tracked folders (+ trx.mat, feat.mat)	Awaiting processing
`2_processed`	Fully processed folders (+ MP4 videos)	Complete
`exp_results`	Result .mat files organized by protocol	Analysis ready
`exp_figures`	Overview figures (PDF/PNG)	QC and review

All network directories reside on the Janelia network share at `\\prfs.hhmi.org\reiser-lab\oaky-cokey\data\`.

Completeness check:

A folder is considered "complete" (ready to be advanced to the next stage) when it contains at least one `.ufmf` file, at least one `.mat` file, and a file whose name starts with `stamp_log`. This ensures the experiment has finished writing all outputs before any copying or processing begins.

3.2 Stage 1: Monitoring and Copying

3.2.1 `monitor_and_copy.py`

Purpose: Watches the acquisition PC's data folder for new experiment directories and copies completed experiments to the network `0_unprocessed` directory.

Where it runs: Acquisition PC (Windows, user `labadmin`). This is a different physical machine and user account from the processing scripts.

Mechanism:

1. Uses the Python `watchdog` library to create a filesystem `Observer` that monitors the acquisition data folder (see paths below) recursively for new directories
2. When a new directory is detected (`on_created` event), it is added to a `pending_folders` set
3. Every 30 seconds, the script iterates through pending folders and checks completeness via `is_folder_complete()`
4. Complete folders are copied to the network using `shutil.copytree()` with `dirs_exist_ok=True`, preserving the full relative path from the source root (so the `{date}/{protocol}/{strain}/{sex}/{time}/` hierarchy is maintained)

5. The script runs indefinitely until manually stopped (Ctrl+C)

Paths:

- Source: `C:\MatlabRoot\FreeWalkOptomotor\data`
- Destination: `\\prfs.hhmi.org\reiserlab\oaky-cokey\data\0_unprocessed`

Logging: `monitor_and_copy.log` in the script directory.

Launcher: `run_monitor_and_copy.bat` sets the Python path and working directory. Designed to be registered as a Windows Task Scheduler task that runs on login.

3.3 Stage 2: Automated Tracking

3.3.1 `monitor_and_track.py`

Purpose: Polls the network `0_unprocessed` directory for completed experiment folders, copies them locally, runs MATLAB FlyTracker batch tracking, and on success moves the tracked data to `1_tracked` on the network and the local archive.

Where it runs: Processing PC (Windows, user `burnettl`).

Mechanism: Polling-based (not watchdog). Scans the network directory every 5 minutes (`SCAN_INTERVAL = 300`). Has an inactivity timer: 15 consecutive scan cycles with no new data triggers auto-exit (15 x 5 min = 75 minutes of inactivity). The counter resets to 0 whenever new data is processed.

Processing flow for each untracked folder:

1. `has_been_processed()` – checks if the folder already exists in `1_tracked` or `2_processed` on the network. Skips if found.
2. `is_folder_complete()` – same completeness check as Stage 1 (.ufmf + .mat + stamp_log).
3. `copy_to_local()` – copies from network `0_unprocessed` to local `00_unprocessed` (avoids tracking over the network, which would be extremely slow).
4. `run_matlab_tracking()` – invokes MATLAB in batch mode:

```
matlab -batch "cd('<folder>'); batch_track_ufmf('<folder>')"
```

5. `tracking_successful()` – walks the folder tree looking for `trx.mat`. Its presence confirms FlyTracker completed successfully.
6. `move_to_tracked()` – performs four operations: (a) copy to local `01_tracked` archive, (b) move from local `00_unprocessed` to network `1_tracked`, (c) clean up empty parent directories locally, (d) delete the original from network `0_unprocessed` and clean up empty parents.

Path constants:

Variable	Path
GROUP_DRIVE_PATH	\\prfs.hhmi.org\...\data\0_unprocessed
LOCAL_PATH	C:\...\oakey-cokey\DATA\00_unprocessed
TRACKED_PATH	\\prfs.hhmi.org\...\data\1_tracked
TRACKED_LOCAL_PATH	C:\...\oakey-cokey\DATA\01_tracked
PROCESSED_PATH	\\prfs.hhmi.org\...\data\2_processed

Launcher: `run_monitor_and_track.bat` creates a dated log file in `monitor_and_track_logs/`.

3.3.2 `batch_track_ufmf.m`

Purpose: Batch-processes all UFMF video files in a given experiment folder using FlyTracker's `simple_noninteractive_flytracker()`.

Location: `tracking/batch_track_ufmf.m`

How it works:

1. Recursively finds all `.ufmf` files under the input folder
2. Loads the base calibration file from `tracking/calibration.mat`
3. For each video folder:
   - Reads the experiment's `LOG.mat` to get `LOG.meta.n_flies`
   - Updates the calibration struct with the actual fly count
   - Saves a per-experiment `calibration.mat` in the video folder
   - Calls `simple_noninteractive_flytracker()` with the video and calibration
4. Produces `trx.mat` (trajectories) and `feat.mat` (features) within each video's output folder
5. Saves a tracking-time log table to `tracking_log/Tracked_{date}.mat`

Tracking options:

Option	Value	Description
`num_chunks`	16	Parallel processing chunks
`num_cores`	8	CPU cores to use
`max_minutes`	Inf	No time limit
`save_JAABA`	1	Save JAABA-compatible output
`save_seg`	0	Don't save segmentation
`force_calib`	1	Force calibration usage
`do_recompute_tracking`	1	Recompute if output exists

Other files in `tracking/`:

- `calibration.mat` – base FlyTracker calibration (arena geometry, background model, detection thresholds)
- `batch_track_ufmf_p27.m` – legacy version that only processes `protocol_27` folders (no longer used by automation)
- `export_tracks_stim_only.m` – utility to extract tracking data for specific frame ranges
- `retrack_by_date_with_calib_file.m` – manual retracking script for specific date ranges

3.4 Stage 3: Daily Processing

3.4.1 `daily_processing.py`

Purpose: Processes all tracked-but-unprocessed date folders by running MATLAB `process_freely_walking_data()`, then distributes results and figures to the network.

Where it runs: Processing PC (Windows, user `burnettl`).

Mechanism: One-shot execution (not a polling loop). Runs once and exits. Designed to be scheduled once daily via Windows Task Scheduler.

Processing flow:

1. `list_date_folders()` – scans `LOCAL_TRACKED` for directories matching the date pattern `YYYY_MM_DD` (any year) using the regular expression `^\d{4}_\d{2}_\d{2}$`.
2. Compares tracked dates against `LOCAL_PROCESSED` to identify unprocessed dates.
3. For each new date: calls `run_matlab_function()` which invokes:

```
matlab -batch "process_freely_walking_data('YYYY_MM_DD')"
```

4. `has_results_for_date()` – searches the results directory for any file starting with the date string.
5. `copy_results_to_network()` – copies result `.mat` files to the network `exp_results` directory, preserving protocol folder structure.
6. `copy_figs_to_network()` – copies `.pdf` and `.png` figure files to the network `exp_figures\overview_figs` directory.
7. `move_folder()` – moves date folder from `01_tracked` to `02_processed` (both local and network). **Warning: if the destination folder already exists, `shutil.move` creates a nested structure – see Section 11.**
8. `copy_mp4s_to_network()` – copies generated `.mp4` video files to network `2_processed`.

Path constants:

Variable	Path
LOCAL_TRACKED	C:\...\oakey-cokey\DATA\01_tracked
LOCAL_PROCESSED	C:\...\oakey-cokey\DATA\02_processed
NETWORK_TRACKED	\\prfs.hhmi.org\...\data\1_tracked
NETWORK_PROCESSED	\\prfs.hhmi.org\...\data\2_processed
LOCAL_RESULTS_ROOT	C:\...\oakey-cokey\results
NETWORK_RESULTS_ROOT	\\prfs.hhmi.org\...\oaky-cokey\exp_results
LOCAL_FIGS_ROOT	C:\...\oakey-cokey\figures\overview_figs
NETWORK_FIGS_ROOT	\\prfs.hhmi.org\...\exp_figures\overview_figs

Launcher: `run_daily_processing.bat` creates a dated log file in `processing_logs/`.

3.5 Supporting Scripts

3.5.1 `reprocessing_script.py`

A variant of `daily_processing.py` designed for **reprocessing ALL dates** in `01_tracked`, regardless of whether they have been processed before. Used when processing logic changes and all data needs to be regenerated.

Key differences from `daily_processing.py`:

- Processes every date folder (does not skip dates already in `02_processed`)
- The `move_folder()` function uses `shutil.rmtree(dst)` to delete the existing destination before moving, avoiding the nesting bug but at the cost of destroying existing data

3.5.2 `copy_movies_to_network.py`

A standalone utility that syncs `.mp4` video files from local `02_processed` to network `2_processed`. Useful for catching up on video transfers when the daily processing pipeline skipped the copy step or when video recording failed for some experiments. Logs folders with missing videos to `missing_videos_log.txt`.

3.6 Configuration and Deployment

Physical machine layout:

Machine	Role	Windows User	Scripts
Acquisition PC	Runs experiments, captures video	`labadmin`	`monitor_and_copy.py`
Processing PC	FlyTracker tracking, MATLAB processing	`burnettl`	`monitor_and_track.py`, `daily_processing.py`

Windows Task Scheduler setup:

- `run_monitor_and_copy.bat` should run continuously on the acquisition PC (triggered on login). It monitors indefinitely until stopped.
- `run_monitor_and_track.bat` should run on the processing PC, triggered after expected experiment completion times. It auto-exits after 75 minutes of inactivity.
- `run_daily_processing.bat` should run once daily on the processing PC, after tracking is expected to finish. It executes once and exits.

Dependencies:

- Python 3.13 (standalone installation on the acquisition PC; the pixi environment can also be used on the processing PC)
- `watchdog` library (required by `monitor_and_copy.py`; listed in `pixi.toml` and also installed via `pip` on the acquisition PC)
- MATLAB with FlyTracker (`simple_noninteractive_flytracker`) on the system PATH
- Network drive `\\prfs.hhmi.org\reiserlab\oaky-cokey\` must be accessible from both PCs

Known Limitations

The automation scripts were developed for rapid deployment and contain several hardcoded assumptions. See Section 11 for a complete list. The most impactful remaining issues are: the lack of a shared configuration file (all paths are duplicated across scripts), the `move_folder()` nesting bug in `daily_processing.py`, and the absence of retry logic for network operations.

4 Running an Experiment – Protocol Functions

At a Glance

- A protocol file (e.g., `protocol_27.m`) orchestrates the entire experiment
- 14 shared functions in `protocols/protocol_functions/` handle initialization, stimulus, and acclimation
- Each experiment has: acclimation, flashes, 2 repetitions of all conditions, final acclimation
- All timing (frame counts and timestamps) is logged in the LOG struct
- Temperature is recorded at the start and end of every phase

4.1 Overview of Protocol Structure

A protocol file (e.g., `protocol_27.m`) is a MATLAB script that orchestrates the entire experiment. It defines the stimulus conditions, calls initialization functions, presents stimuli in randomized order, and saves all timing data.

The protocol calls functions from the `protocols/protocol_functions/` directory. There are 14 functions in this directory, organized into four categories: initialization, stimulus presentation, acclimation, and data export.

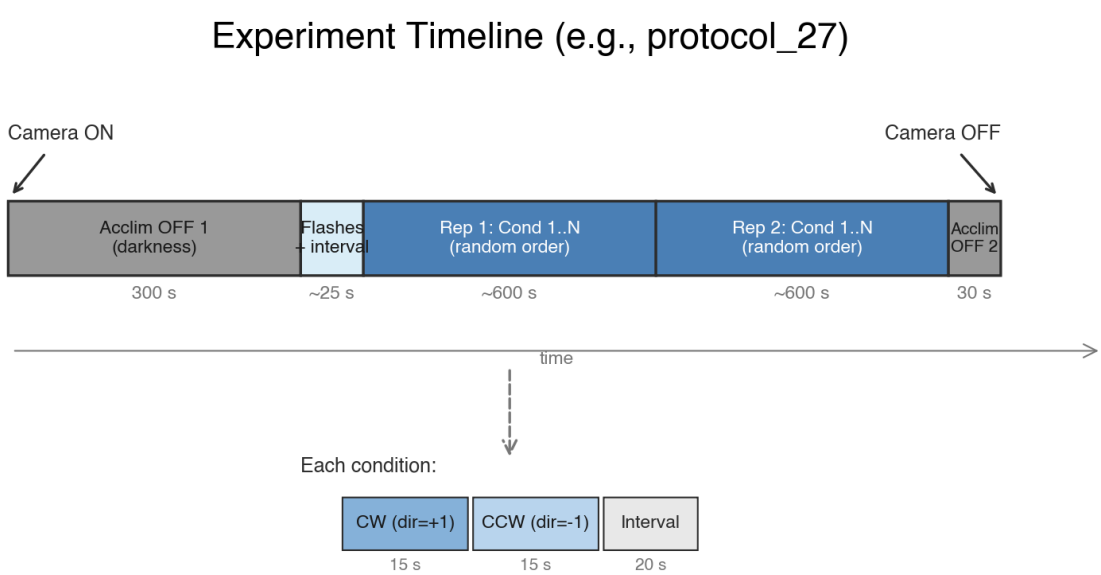

Figure 3: Experiment timeline showing the temporal structure of a typical protocol. Each condition block consists of two trials (CW and CCW) plus an interval.

4.1.1 Typical Experiment Flow (protocol_27.m)

1. Initialize camera, folders, and metadata (`initialize_video_and_folders`)
2. Initialize temperature recording (`initialize_temp_recording`)
3. Start camera capture
4. **Acclimation OFF 1** – 300s of recording in complete darkness (`present_acclim_off`, rep=1)
5. **Full-field flashes** – 5s calibration flashes (pattern 48), then 20s dark interval
6. **Repetition 1** – Present all conditions once in random order
7. **Repetition 2** – Present all conditions again in a new random order
8. **Acclimation OFF 2** – 30s of dark recording (`present_acclim_off`, rep=2)
9. Stop camera, save LOG.mat, export metadata to Google Sheets

Each condition consists of:

- 2 stimulus trials (15s each, alternating CW and CCW directions)
- 1 interval (20s, dark background pattern)

4.2 Initialization Functions

4.2.1 `get_input_parameters.m`

Purpose: Opens a GUI dialog to capture experimental metadata before each experiment.

Inputs: None

Outputs: `selections` struct with fields:

Field	Options	Default
`nFlies`	1–20	15
`Experimenter`	GraceBiondi, AparnaDev, LauraBurnett, other	GraceBiondi
`Strain`	28 strain options (csw1118, jfrc100_es_shibire_kir, RNAi controls, etc.)	jfrc100_es_shibire_kir
`Age`	1–8, >8, age ranges (1-2, 2-3, …), NaN	4
`Sex`	F, M, NaN	F
`LightCycle`	20:00_12:00, 01:00_17:00, NaN	01:00_17:00

This metadata is stored in the LOG struct and exported to Google Sheets for experiment tracking.

4.2.2 `initialize_video_and_folders.m`

Purpose: Sets up the camera, creates the folder hierarchy for data storage, and initializes the LOG struct.

Inputs:

- `project_data_folder` – root data directory (e.g., `C:\MatlabRoot\FreeWalkOptomotor\data`)
- `func_name` – protocol name (e.g., `"protocol_27"`)

Outputs:

- `LOG` – metadata struct (with `LOG.meta` subfield)
- `vidobj` – BIAS camera interface object
- `exp_folder` – full path to experiment folder
- `date_str`, `t_str` – date and time strings
- `params` – augmented parameter struct

What it does:

1. Calls `get_input_parameters()` to collect metadata via GUI
2. Connects to the BIAS camera on `localhost:5010`
3. Creates a nested folder hierarchy:

```
data/{date}/{protocol}/{strain}/{sex}/{time}/
```

4. Configures video recording (UFMF format, prefix `REC_`)
5. Creates `LOG.meta` with fields: `date`, `time`, `func_name`, `fly_strain`, `fly_age`, `fly_sex`, `light_cycle`, `experimenter`, `n_flies`
6. Prompts for start-of-experiment notes

4.2.3 `initialize_temp_recording.m`

Purpose: Configures a National Instruments DAQ for temperature monitoring.

Inputs: None

Outputs: `d` – DAQ session object

Configuration:

- Device: cDAQ1Mod1 (National Instruments CompactDAQ)
- Channel ai0: `outside_probe` (K-type thermocouple) – ambient temperature
- Channel ai1: `ring_probe` (K-type thermocouple) – arena ring temperature
- Sampling rate: 2 Hz

4.2.4 `get_temp_rec.m`

Purpose: Reads a single temperature measurement from both probes.

Inputs: `d` – DAQ session object

Outputs: `t_outside` (ambient), `t_ring` (arena ring) – temperature in degrees C

How it works: Reads 1 second of data from the DAQ and extracts the first sample from each channel. Called at the start and end of each stimulus condition and acclimation period.

4.3 Stimulus Presentation Functions

All stimulus presentation functions share a common pattern:

1. Record temperature at start
2. Extract parameters from the `all_conditions` matrix row
3. Present stimulus trials (varying by function type)
4. Present interval pattern

5. Record temperature at end
6. Return a Log struct with timing and parameter data

4.3.1 The `all_conditions` Matrix

Each protocol defines an `all_conditions` matrix with columns:

Column	Field	Description
1	pattern_id	LED pattern number for the stimulus
2	interval_id	LED pattern for the interval (usually 47 = dark)
3	speed_patt	Stimulus animation speed (0–127)
4	speed_int	Interval animation speed (usually 0 or 1)
5	trial_dur	Duration of each trial in seconds
6	int_dur	Duration of the interval in seconds
7	condition_n	Condition number (for identification)

Speed values:

Speed Value	Approximate Rate
0	Static (no motion)
1	Minimal (fixation)
8	~4 Hz
32	~16 Hz
64	~32 Hz
127	Maximum (~63 Hz)

4.3.2 Variant Comparison

All six `present_optomotor_stimulus` variants share the same signature and core structure: extract condition parameters, run stimulus trials with `Panel_com`, and log frame timestamps and temperatures. The table below summarises their key differences.

Variant	Trials	Direction	Interval	Swap?	Special Feature
`(base)`	2	Alternates ± 1	After all trials	No	`t_stim = trial_len` $\times$ `2`
`_w_interval`	Col. 7	+1 then -1	After every trial	No	Col. 7 = `num_trials`
`_one_direction`	1	+1 only	After the trial	No	Simplest variant
`_curtain`	15 or 2	Alternates ± 1	After all trials	Yes	Swaps paired pattern on trial 2
`_curtain_w_interval`	Col. 7	+1 then -1	After every trial	Yes	Curtain swap + per-trial intervals
`_diff_contrasts`	2	Alternates ± 1	After all trials	No	Y-axis selects contrast level

Key notes:

- **Interval placement** is the biggest structural split. The `_w_interval` variants log the interval as its own trial entry inside the loop, producing longer arrays that alternate stimulus/interval.
- **Curtain variants** load a paired pattern on trial 2 via a hardcoded `switch` block because curtain patterns are directionally asymmetric.
- **`_diff_contrasts`** is unique in using the pattern's **y-axis** (`set_position([1, which_condition])`) to index into contrast levels, while all others use `[1, 1]`.

- **Column 7 meaning changes:** In `_w_interval` and `_curtain_w_interval`, column 7 of `all_conditions` is `num_trials`. In all other variants, column 7 is `which_condition`.

4.3.3 `present_optomotor_stimulus.m` – Standard Bidirectional

Purpose: Presents a moving grating pattern in two directions (CW then CCW), followed by a dark interval.

Inputs: `current_condition` (index), `all_conditions` (matrix), `vidobj` (camera), `d` (DAQ)

Outputs: `Log` struct

Step-by-step:

1. Record start temperature
2. Extract condition parameters from `all_conditions(current_condition, :)`
3. Calculate `num_trials = 2` (always: `t_stim = trial_len * 2`, then `num_trials = t_stim / trial_len`)
4. Load the stimulus pattern via `Panel_com('set_pattern_id', pattern_id)`
5. **Trial loop** (2 iterations):
   - Alternate direction: `dir_val` flips between +1 and -1 each trial
   - Set speed and direction: `Panel_com('send_gain_bias', [speed * dir_val, 0, 0, 0])`
   - Start the LED panels: `Panel_com('start')`
   - **Log `start_f` and `start_t`** from the camera (`vidobj.getFrameCount()` and `vidobj.getTimeStamp()`)
   - Pause for `trial_len` seconds (the stimulus runs during this pause)
   - Stop the panels: `Panel_com('stop')`
   - **Log `stop_f` and `stop_t`** from the camera
6. **Interval phase:**
   - Load interval pattern (pattern 47, dark background)
   - Set direction to 0 (no motion)
   - Log start/stop frame counts and timestamps
   - Pause for `interval_dur` seconds
7. Record end temperature

Log structure returned:

- `Log.trial` = [1, 2, 3] – trial indices
- `Log.dir` = [+1, -1, 0] – directions (CW, CCW, interval)
- `Log.start_t`, `Log.start_f` = [t1, t2, t3] – timestamps and frame counts at onset
- `Log.stop_t`, `Log.stop_f` = [t1, t2, t3] – timestamps and frame counts at offset
- Plus: `trial_len`, `interval_dur`, `num_trials`, `optomotor_pattern`, `interval_pattern`, `optomotor_speed`, `interval_speed`, `which_condition`, and four temperature readings

4.3.4 `present_optomotor_stimulus_w_interval.m` – Per-Trial Intervals

Difference from standard: Instead of one interval after all trials, this function inserts a dark interval **after each trial**. This creates a log with alternating stimulus and interval entries.

The `num_trials` value comes directly from `all_conditions` column 7 (not derived from trial length). First trial direction is +1, subsequent trials use -1.

Log structure: Arrays are longer, alternating between stimulus entries (dir = +1 or -1) and interval entries (dir = 0).

4.3.5 `present_optomotor_stimulus_one_direction.m` – Single Direction

Difference from standard: Only presents one trial (`num_trials = 1`) in one direction (dir = +1), followed by an interval. Used for stimuli where bidirectional testing is not needed.

4.3.6 `present_optomotor_stimulus_curtain.m` – Curtain Patterns

Difference from standard: Curtain stimuli (progressive ON/OFF edges) require a different pattern for each direction. This function uses a hardcoded pattern-pair mapping:

Forward Pattern	Reverse Pattern
19	20
51	52
53	54
55	56

On trial 1, the original pattern is used. On trial 2, the pattern switches to the complementary pattern from the table above.

Number of trials depends on trial length: if `trial_len == 2`, then 15 trials; if `trial_len == 15`, then 2 trials.

4.3.7 `present_optomotor_stimulus_curtain_w_interval.m`

Combines the curtain pattern-switching logic with per-trial interval logging (like `_w_interval` above).

4.3.8 `present_optomotor_stimulus_diff_contrasts.m` – Contrast Variation

Difference from standard: Uses the `which_condition` parameter as a y-position index to select different contrast levels from the same pattern:

```
Panel_com('set_position', [1 which_condition])
```

The contrast levels are: [0.11, 0.2, 0.333, 0.4, 0.556, 0.75, 1.0].

Additionally logs `Log.contrast` with the contrast value for the current condition.

4.3.9 `present_fixation_stimulus.m` – Static Bar Display

Purpose: Presents a stationary pattern (no motion) for phototaxis/bar fixation experiments.

Key differences:

- Speed is set to `[0 0 0 0]` (no animation)
- Position is set to `[1 1]` (first frame only – static display)
- Single trial, no direction alternation

4.3.10 `present_shifted_stimulus.m` – Open-Loop Shifted Patterns

Key differences:

- Explicitly sets controller mode to `[0 0]` (double open-loop)
- Does **not** take a DAQ device parameter (no temperature recording within the function)
- Uses `Panel_com('all_off')` for intervals instead of showing an interval pattern
- Includes a 30-second "ALL OFF" phase after stimulus trials

4.3.11 `present_acclim_off.m` – Dark Acclimation

Purpose: Records fly behavior in complete darkness (all LEDs off). Called twice per experiment.

Inputs: `LOG` (modified in place), `vidobj`, `t_pause`, `t_acclim` (duration), `rep` (1 or 2), `d` (DAQ)

Outputs: Modified `LOG` with `acclim_off1` or `acclim_off2` added

Step-by-step:

1. Record start temperature
2. Turn off all LEDs: `Panel_com('all_off')`
3. Log start frame count and timestamp
4. Pause for `t_acclim` seconds
5. Log stop frame count and timestamp
6. Record end temperature
7. Store in `LOG.acclim_off1` (if rep=1) or `LOG.acclim_off2` (if rep=2)

4.4 Export Function

4.4.1 `export_to_google_sheets.m`

Purpose: Pushes experiment metadata to a Google Sheet via HTTP POST for centralized experiment tracking.

Inputs: `params` struct with: Date, Time, Experimenter, Strain, Age, Sex, LightCycle, Protocol, NotesStart, NotesEnd

How it works: Submits the parameters to a Google Form response endpoint using MATLAB's `webwrite()`. Each field maps to a specific Google Form entry ID. The data appears in a linked Google Sheet, providing a lab-wide experiment log.

5 The LOG Data Structure

At a Glance

- The LOG struct is built progressively during the experiment – each phase adds its subfield
- `LOG.meta` stores experiment metadata (strain, age, sex, timing parameters, etc.)
- `LOG.acclim_off1` and `LOG.acclim_off2` store dark acclimation timing
- `LOG.log_1` through `LOG.log_N` store per-condition stimulus timing with frame counts
- Frame counts (`start_f`, `stop_f`) are the critical link to the video data

5.1 How the LOG Is Built

The LOG struct is progressively assembled during an experiment. Each phase of the protocol adds its own subfield.

LOG Data Structure

```
LOG
 ├── meta
 │    ├── date, time, func_name
 │    ├── fly_strain, fly_age, fly_sex                                  Experiment
 │    ├── light_cycle, experimenter, n_flies                            metadata
 │    ├── t_acclim_start, t_flash, t_acclim_end
 │    ├── start_temp_outside, start_temp_ring
 │    ├── end_temp_outside, end_temp_ring
 │    ├── cond_array          [num_cond x 7 matrix]
 │    └── random_order        [1 x num_cond vector]
 │
 ├── acclim_off1
 │    ├── condition (=0), dir (=0)
 │    ├── start_t, start_f    [camera timestamp & frame]               Dark
 │    ├── stop_t, stop_f      [camera timestamp & frame]               acclimation
 │    └── t_outside_start, t_ring_start, t_outside_end, t_ring_end
 │
 ├── acclim_patt
 │    ├── flash_pattern (=48), flash_speed, flash_dur, dir (=0)        Calibration
 │    ├── start_t, start_f, stop_t, stop_f    [flash period]           flashes
 │    └── start_t_int, start_f_int, stop_t_int, stop_f_int  [interval]
 │
 ├── log_1 ... log_N     (N = 2 x num_conditions)
 │    ├── trial[]             [1, 2, 3]  (trial index per entry)
 │    ├── dir[]               [+1, -1, 0]  (CW, CCW, interval)
 │    ├── start_t[], start_f[] [timestamps & frames at onset]
 │    ├── stop_t[], stop_f[]   [timestamps & frames at offset]         Stimulus
 │    ├── trial_len, interval_dur, num_trials                          conditions
 │    ├── optomotor_pattern, interval_pattern
 │    ├── optomotor_speed, interval_speed
 │    ├── which_condition     [condition number from all_conditions]
 │    └── t_outside_start, t_ring_start, t_outside_end, t_ring_end
 │                                                                     Post-stimulus
 │                                                                     acclimation
 └── acclim_off2
      └── (same fields as acclim_off1)
```

Figure 4: The LOG data structure showing all fields organized by experiment phase.

Build sequence:

1. `initialize_video_and_folders` creates `LOG.meta` with date, time, strain, age, sex, etc.
2. `present_acclim_off(LOG, ..., 1, d)` adds `LOG.acclim_off1`
3. The protocol script adds `LOG.acclim_patt` (calibration flashes)
4. The condition loop adds `LOG.log_1`, `LOG.log_2`, … `LOG.log_N` (where N = 2 x num_-conditions, covering both repetitions)
5. `present_acclim_off(LOG, ..., 2, d)` adds `LOG.acclim_off2`
6. The protocol script appends final fields to `LOG.meta`: timing parameters, start/end temperatures, `cond_array`, `random_order`
7. The LOG is saved as `LOG_{date}_{time}.mat`

5.2 LOG.meta Fields

Field	Type	Description
`date`	datetime	Experiment date (yyyy_MM_dd)
`time`	datetime	Experiment start time
`func_name`	string	Protocol name (e.g., “protocol_27”)
`fly_strain`	string	Genotype of the flies
`fly_age`	string	Age in days
`fly_sex`	string	“F” or “M”
`light_cycle`	string	Light/dark cycle timing
`experimenter`	string	Person running the experiment
`n_flies`	string	Number of flies in the arena
`t_acclim_start`	double	Duration of pre-stimulus acclimation (seconds)
`t_flash`	double	Duration of calibration flashes (seconds)
`t_acclim_end`	double	Duration of post-stimulus acclimation (seconds)
`start_temp_outside`	double	Ambient temperature at experiment start
`start_temp_ring`	double	Arena ring temperature at experiment start
`end_temp_outside`	double	Ambient temperature at experiment end
`end_temp_ring`	double	Arena ring temperature at experiment end
`cond_array`	matrix	The full `all_conditions` matrix [N x 7]
`random_order`	vector	Randomized condition presentation order

5.3 LOG.acclim_off1 and LOG.acclim_off2 Fields

Field	Type	Description
`condition`	double	Always 0 (darkness)
`dir`	double	Always 0 (no direction)
`start_t`	double	Camera timestamp at acclimation start
`start_f`	double	Camera frame count at acclimation start
`stop_t`	double	Camera timestamp at acclimation end
`stop_f`	double	Camera frame count at acclimation end
`t_outside_start`	double	Ambient temperature at start
`t_ring_start`	double	Ring temperature at start

Field	Type	Description
`t_outside_end`	double	Ambient temperature at end
`t_ring_end`	double	Ring temperature at end

5.4 LOG.acclim_patt Fields

Field	Type	Description
`flash_pattern`	double	Pattern 48 (full-field flash)
`flash_speed`	double	Flash animation speed
`flash_dur`	double	Flash duration in seconds
`dir`	double	Always 0
`start_t`, `start_f`	double	Camera timing at flash start
`stop_t`, `stop_f`	double	Camera timing at flash end
`start_t_int`, `start_f_int`	double	Camera timing at interval start
`stop_t_int`, `stop_f_int`	double	Camera timing at interval end

5.5 LOG.log_N Fields (Stimulus Conditions)

Each `LOG.log_1` through `LOG.log_N` contains:

Field	Type	Description
`trial`	array	Trial indices [1, 2, 3]
`dir`	array	Directions [+1, -1, 0] (CW, CCW, interval)
`start_t`	array	Camera timestamps at each trial/interval onset
`start_f`	array	Camera frame counts at each trial/interval onset
`stop_t`	array	Camera timestamps at each trial/interval offset
`stop_f`	array	Camera frame counts at each trial/interval offset
`trial_len`	double	Trial duration in seconds
`interval_dur`	double	Interval duration in seconds
`num_trials`	double	Number of stimulus trials (usually 2)
`optomotor_pattern`	double	Pattern ID used for stimulus
`interval_pattern`	double	Pattern ID for interval (usually 47)
`optomotor_speed`	double	Stimulus speed (0–127)
`interval_speed`	double	Interval speed (usually 0 or 1)
`which_condition`	double	Condition number from `all_conditions`
`t_outside_start`	double	Ambient temperature at start
`t_ring_start`	double	Ring temperature at start
`t_outside_end`	double	Ambient temperature at end
`t_ring_end`	double	Ring temperature at end

Important: For the standard `present_optomotor_stimulus`, the arrays have 3 entries (trial 1, trial 2, interval). For `_w_interval` variants, arrays alternate: stimulus, interval, stimulus, interval, …

5.6 Frame Synchronization

The critical link between stimulus events and behavioral data is the **frame count**. Each time a stimulus starts or stops, the protocol records the camera's current frame number. This allows precise alignment during offline analysis.

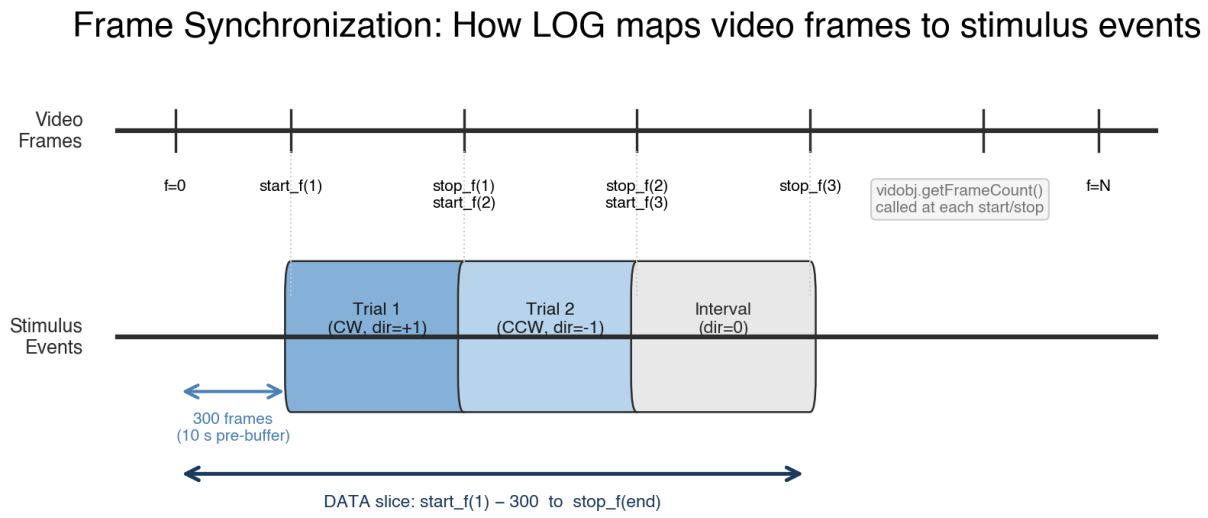

Figure 5: How LOG frame counts (start_f, stop_f) map video frames to stimulus events. The 300-frame pre-buffer is included when slicing data per condition.

Key points:

- `start_f = vidobj.getFrameCount().value` is called **immediately after** `Panel_com('start')` – this is the frame when the stimulus becomes visible
- `stop_f = vidobj.getFrameCount().value` is called **immediately after** `Panel_com('stop')` – this is the frame when the stimulus disappears
- At 30 fps, each frame is ~33.3 ms
- During processing, a **300-frame (10-second) pre-buffer** is included before each condition to capture baseline behavior before stimulus onset

6 Data Processing Pipeline

At a Glance

- `process_freely_walking_data` iterates through all experiments for a given date
- `process_data_features` handles one experiment: loads tracking, computes metrics, saves results
- `combine_data_one_cohort` is the core function: filters bad tracking, computes 12 behavioral metrics
- `comb_data_one_cohort_cond` slices continuous data into per-condition segments using LOG frame indices
- `comb_data_across_cohorts_cond` merges all experiments for group-level analysis

6.1 `process_freely_walking_data.m` – The Orchestrator

This is the main entry point for processing all experiments from a given date. It is called automatically by `daily_processing.py` when the automated pipeline (Section 3) is active, or can be invoked manually from the MATLAB command line.

Usage:

```
process_freely_walking_data("2024_09_24")
process_freely_walking_data("2024_09_24", true)  % with stimulus videos
```

How it works:

1. Sets the project root and data paths
2. Navigates to the date folder in `DATA/01_tracked/{date}/`
3. Iterates through all protocol folders (`protocol_*`)
4. For each protocol, iterates through strain folders
5. For each strain, iterates through sex folders (F/M)
6. Calls `process_data_features()` for each combination

Data Folder Structure

```
Raw Data (during experiment):
─────────────────────────────
C:\MatlabRoot\FreeWalkOptomotor\data\
  └── 2024_09_24\                          ← date
       └── protocol_27\                     ← protocol
            └── jfrc100_es_shibire_kir\      ← strain
                 └── F\                       ← sex
                      └── 11_06_44\            ← time (HH_MM_SS)
                           ├── LOG_2024_09_24_11_06_44.mat
                           └── REC_\
                                └── movie\
                                     ├── movie.ufmf   (video)
                                     └── movie_JAABA\
                                          └── trx.mat
```

```
After FlyTracker processing:
────────────────────────────
REC_\movie\ also contains:
  ├── movie-feat.mat                       ← FlyTracker features
  └── movie_JAABA\trx.mat                  ← Trajectory data
```

```
Processed Results:
──────────────────
results\
  └── protocol_27\
       └── jfrc100_es_shibire_kir\
            └── F\
                 └── 2024-09-24_11-06-44_..._data.mat
```

Figure 6: Data folder hierarchy showing the organization of raw, tracked, and processed data.

6.2 `process_data_features.m` – Single Experiment Processing

Processes a single experiment: loads tracking data, computes metrics, generates overview plots, and saves results.

Step-by-step:

1. **Load LOG** – Finds `LOG_*.mat` in the experiment folder
2. **Load FlyTracker output** – Navigates into `REC_*/movie/` and loads:
   - `feat` from `movie-feat.mat` (behavioral features)
   - `trx` from `movie_JAABA/trx.mat` (trajectory data)
3. **Compute behavioral metrics** – Calls `combine_data_one_cohort(feat, trx)` to produce `comb_data`
4. **Split by condition** – Calls `comb_data_one_cohort_cond(LOG, comb_data, protocol)` to produce a per-condition DATA struct

5. **Generate overview plots** – Histograms of locomotor parameters, full-experiment feature timeseries, and per-condition timeseries for forward velocity, angular velocity, turning rate, and distance from center
6. **Save** – Writes `{date}_{time}_{strain}_{protocol}_{sex}_data.mat` containing: `LOG`, `feat`, `trx`, `comb_data`, `n_fly_data`

6.3 `combine_data_one_cohort.m` – Core Feature Extraction

This is the most important processing function. It takes raw FlyTracker output and computes all derived behavioral metrics.

Inputs: `feat` (FlyTracker features struct), `trx` (trajectory struct)

Outputs: `comb_data` (struct with 12 metric matrices), modified `feat` and `trx` (bad flies removed)

6.3.1 Step 1: Remove Bad Tracking (`check_tracking_FlyTrk`)

FlyTracker sometimes produces artifacts: a single fly split into two tracks, or a non-fly object (dust, shadow) tracked. The function `check_tracking_FlyTrk` detects these:

1. Extract the frame count for each fly: `d = [trx.nframes]`
2. Compute the mode (most common frame count) – this is the true video length
3. Any fly whose frame count differs from the mode is flagged for removal
4. Flagged flies are removed from both `trx` and `feat.data`

This is a simple but effective quality control step.

6.3.2 Step 2: High-Velocity Filtering

For each fly, frames where the FlyTracker-reported velocity exceeds 50 mm/s are marked as tracking errors. At these frames, position (x, y), heading, and distance-to-wall data are set to NaN for interpolation.

50 mm/s is well above the typical maximum walking speed of *Drosophila* (~30 mm/s) and reliably catches tracking jumps.

6.3.3 Step 3: Interpolation

Missing values (from filtering) are filled using method-appropriate interpolation:

Data	Method	Reason
`d_wall_data`	Spline	Smooth, continuous changes in distance
`heading_data`	Previous value	Avoids introducing artificial heading jumps
`x_data`, `y_data`	Spline	Smooth position trajectories

6.3.4 Step 4: Distance from Center

```
dist_data = 120 - d_wall_data;
```

FlyTracker reports `d_wall` (distance from fly to arena wall). The arena radius is 120 mm, so distance from center is simply `120 - d_wall`.

6.3.5 Step 5: Angular Velocity (`vel_estimate`)

Angular velocity measures how fast the fly is turning, in degrees per second.

Method: Least-squares line fit to heading data in a sliding window.

```
av_data_rad = vel_estimate(heading, samp_rate, 'line_fit', t_window, []);
av_data = rad2deg(av_data_rad);
```

Parameters:

- `samp_rate = 1/30` (30 fps)
- `t_window = 16` – the sliding window is 16 frames (~0.53 seconds)
- The `vel_estimate` function unwraps heading internally, fits a line in the window, and returns the slope as angular velocity

Larger `t_window` values produce smoother angular velocity estimates.

6.3.6 Step 6: Forward Velocity (`gaussian_conv`)

Forward velocity measures the speed of the fly **in the direction it is heading**, in mm/s.

Computation:

1. Smooth x and y positions with Gaussian convolution
2. Compute frame-to-frame displacements: `dx = diff(x)`, `dy = diff(y)`
3. Convert to velocities: `vx = dx / dt`, `vy = dy / dt`
4. **Project onto heading direction:**

```
fv = vx .* cos(heading) + vy .* sin(heading);
```

5. **Filter:** Negative values set to NaN (fly moving backwards = tracking artifact). Values > 50 mm/s set to NaN.
6. Fill NaNs with linear interpolation
7. Prepend `fv(1)` to maintain array length (since `diff` reduces length by 1)

6.3.7 Step 7: Three-Point Velocity (`calculate_three_point_velocity`)

Speed magnitude regardless of heading direction, computed using central differences:

```
% Forward difference at first point:
vx(1) = (x(2) - x(1)) / dt;

% Central difference for intermediate points:
vx(2:n-1) = (x(3:n) - x(1:n-2)) / (2 * dt);
```

```
% Backward difference at last point:
vx(n) = (x(n) - x(n-1)) / dt;

v = sqrt(vx.^2 + vy.^2);
```

This gives the speed (always positive) without regard to heading direction. Central difference is more accurate than simple forward difference.

6.3.8 Step 8: Turning Rate (Curvature)

Turning rate combines angular and forward velocity:

```
curv_data = av_data ./ fv_data;  % deg/mm
```

Filtering:

- Frames where `abs(fv_data) < 0.1` mm/s are set to NaN (turning rate is undefined when the fly is barely moving)
- Infinite values are set to NaN

Units are degrees per millimeter: how many degrees the fly turns for each millimeter it walks.

6.3.9 Step 9: Viewing Distance (`calculate_viewing_distance`)

The distance from the fly to the arena wall **in the direction it is heading**. This tells us how far ahead the fly can "see" before the wall.

Algorithm: Solves the ray-circle intersection equation.

Given a fly at position (x_f, y_f) heading in direction θ, and a circular arena centered at (x_c, y_c) with radius R:

The ray is parameterized as:

$$x(t) = x_f + t\cos\theta, \quad y(t) = y_f + t\sin\theta$$

Substituting into the circle equation $(x - x_c)^2 + (y - y_c)^2 = R^2$ gives a quadratic in t:

$$At^2 + Bt + C = 0$$

where $A = 1$, $B = 2[(x_f - x_c)\cos\theta + (y_f - y_c)\sin\theta]$, $C = (x_f - x_c)^2 + (y_f - y_c)^2 - R^2$.

The viewing distance is the smallest positive root.

Arena parameters (hardcoded):

- PPM (pixels per mm) = 4.1691
- Center of arena = (528, 520) pixels = (126.6, 124.7) mm
- Arena radius = 496 pixels = 119.0 mm

6.3.10 Step 10: Inter-Fly Distance and Angle (calculate_distance_to_nearest_fly)

For each fly at each frame:

- **IFD_data** (inter-fly distance): Euclidean distance to the nearest other fly, in mm
- **IFA_data** (inter-fly angle): Angle to the nearest other fly relative to the focal fly's heading, in degrees

6.3.11 Output: comb_data Struct

All metrics are stored as [n_flies x n_frames] matrices:

Field	Description	Units
vel_data	Three-point velocity	mm/s
fv_data	Forward velocity	mm/s
av_data	Angular velocity	deg/s
curv_data	Turning rate	deg/mm
dist_data	Distance from center	mm
heading_data	Unwrapped heading	deg
heading_wrap	Wrapped heading	deg
x_data	X position	mm
y_data	Y position	mm
view_dist	Viewing distance to wall	mm
IFD_data	Inter-fly distance	mm
IFA_data	Inter-fly angle	deg

6.4 comb_data_one_cohort_cond.m – Splitting by Condition

This function takes the continuous comb_data matrices and slices them into per-condition segments using the LOG's frame indices.

How it works:

1. Find all LOG fields starting with log_ to determine the number of conditions
2. The first half of conditions belong to **Repetition 1**, the second half to **Repetition 2**
3. For each condition log:
   - Extract which_condition (the condition number from all_conditions)
   - Compute the data slice: start_f(1) - 300 to stop_f(end)
   - The 300-frame (10-second) pre-buffer captures baseline behavior before stimulus onset
   - Store the slice as DATA.(strain).(landing).(sex)(sz).R1_condition_N or R2_condition_N
4. Also store acclimation phases: acclim_off1, acclim_patt, acclim_off2

Condition matching: For newer protocols (protocol_24, protocol_27), the condition number is stored directly in Log.which_condition. For older protocols, the function matches parameters against the protocol's condition array.

6.5 comb_data_across_cohorts_cond.m – Merging Across Experiments

Combines processed data from all experiments for a single protocol into one hierarchical DATA struct.

How it works:

1. Find all *_data.mat files in the protocol results directory
2. For each file, load comb_data, LOG, and n_fly_data
3. Extract strain and sex from LOG.meta
4. Append to the DATA struct, incrementing the cohort index
5. Process each condition using the same frame-slicing logic

This produces the final DATA struct used for group analysis.

7 The DATA Struct

At a Glance

- DATA is the final output: a hierarchical struct organizing all experiments
- Levels: strain, sex, cohort (array index), condition, metric data
- Conditions are named R1_condition_1, R2_condition_N, etc., plus acclimation phases
- Each condition contains stimulus metadata and all 12 behavioral metric matrices
- All data matrices are [n_flies x n_frames]

7.1 Hierarchy

The DATA struct organizes all experimental data in a nested hierarchy:

DATA Struct Hierarchy

```
DATA
 ├── jfrc100_es_shibire_kir                    (strain name)
 │    ├── F                                     (sex)
 │    │    ├── (1)                               (cohort index = experiment 1)
 │    │    │    ├── meta
 │    │    │    │    ├── date, time, func_name, fly_strain, ...
 │    │    │    │    └── n_flies_arena, n_flies, n_flies_rm
 │    │    │    ├── acclim_off1
 │    │    │    │    └── fv_data, av_data, dist_data, ...  [n_flies x n_frames]
 │    │    │    ├── acclim_patt
 │    │    │    │    └── fv_data, av_data, dist_data, ...  [n_flies x n_frames]
 │    │    │    ├── R1_condition_1                     (Rep 1, Condition 1)
 │    │    │    │    ├── trial_len, optomotor_pattern, optomotor_speed
 │    │    │    │    ├── interval_dur, interval_pattern, interval_speed
 │    │    │    │    ├── start_flicker_f
 │    │    │    │    └── fv_data, av_data, curv_data, dist_data,
 │    │    │    │       vel_data, heading_data, heading_wrap,
 │    │    │    │       x_data, y_data, view_dist, IFD_data, IFA_data
 │    │    │    ├── R1_condition_2  ...  R1_condition_N
 │    │    │    ├── R2_condition_1  ...  R2_condition_N    (Rep 2)
 │    │    │    └── acclim_off2
 │    │    │         └── fv_data, av_data, dist_data, ...
 │    │    ├── (2)                               (cohort index = experiment 2)
 │    │    │    └── ...
 │    │    └── (3) ...
 │    └── M                                     (sex)
 │         └── ...
 └── csw1118                                    (another strain)
      └── ...
```

Figure 7: DATA struct hierarchy showing the nesting levels from strain down to individual behavioral metrics.

Example: Accessing DATA

```
% Forward velocity: strain, sex F, cohort 1, Rep 1 condition 1
fv = DATA.jfrc100_es_shibire_kir.F(1).R1_condition_1.fv_data;
%   fv is [n_flies x n_frames]
```

Levels:

1. **Strain** – e.g., `DATA.jfrc100_es_shibire_kir`, `DATA.csw1118`
2. **Sex** – `DATA.(strain).F` or `DATA.(strain).M`
3. **Cohort index** – Array index for each experimental session, e.g., `DATA.(strain).F(1)`,

DATA.(strain).F(2)

4. **Condition** – Named fields for each experimental phase
5. **Data arrays** – `[n_flies x n_frames]` matrices of behavioral metrics

7.2 Condition Names

Name	Description
`meta`	Experiment metadata (same as LOG.meta + fly counts)
`acclim_off1`	Pre-stimulus dark acclimation (300s)
`acclim_patt`	Calibration flash period
`R1_condition_1` … `R1_condition_N`	Repetition 1, conditions 1 through N
`R2_condition_1` … `R2_condition_N`	Repetition 2, conditions 1 through N
`acclim_off2`	Post-stimulus dark acclimation (30s)

Condition numbers match the `condition_n` column in the protocol's `all_conditions` matrix.

7.3 Per-Condition Fields

Each condition contains stimulus metadata and all behavioral metrics:

Stimulus metadata: `trial_len`, `interval_dur`, `optomotor_pattern`, `optomotor_speed`, `interval_pattern`, `interval_speed`, `start_flicker_f`

Behavioral data (all `[n_flies x n_frames]`): `fv_data`, `av_data`, `curv_data`, `dist_data`, `vel_data`, `heading_data`, `heading_wrap`, `x_data`, `y_data`, `view_dist`, `IFD_data`, `IFA_data`

7.4 Accessing Data – Examples

```
% Load combined data for protocol_27
protocol_dir = '/path/to/results/protocol_27';
DATA = comb_data_across_cohorts_cond( ...
    protocol_dir);

% Get forward velocity:
%   strain jfrc100, sex F, cohort 1, R1 cond 1
fv = DATA.jfrc100_es_shibire_kir ...
    .F(1).R1_condition_1.fv_data;
% fv is [n_flies x n_frames]

% Mean forward velocity across all flies
mean_fv = mean(fv, 1);  % [1 x n_frames]

% Get the stimulus parameters
speed = DATA.jfrc100_es_shibire_kir ...
    .F(1).R1_condition_1.optomotor_speed;
pattern = DATA.jfrc100_es_shibire_kir ...
    .F(1).R1_condition_1.optomotor_pattern;
```

```
% Number of cohorts for this strain/sex
n_cohorts = length( ...
    DATA.jfrc100_es_shibire_kir.F);

% List all strains in the dataset
strain_names = fieldnames(DATA);
```

8 Behavioral Metrics Reference

At a Glance

- 12 behavioral metrics are computed for each fly at each frame
- Forward velocity and angular velocity are the primary optomotor response measures
- All metrics are stored as `[n_flies x n_frames]` matrices in `comb_data` and `DATA`
- See the Quick Reference table (Section 1) for a compact summary of all metrics

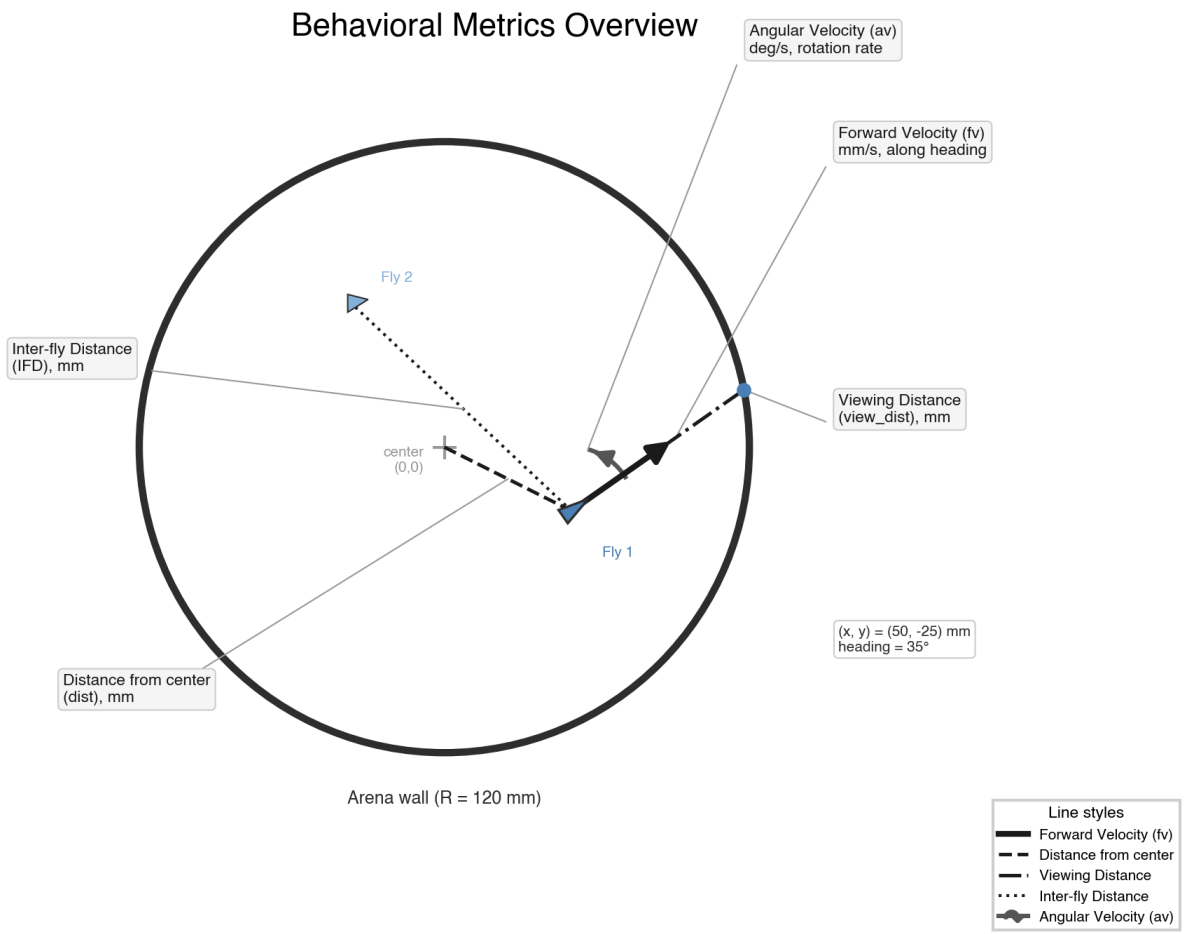

Figure 8: Overview of behavioral metrics measured for each fly in the arena. Each metric captures a different aspect of walking behavior.

8.1 Forward Velocity (`fv_data`)

Property	Value
Units	mm/s
Range	0 – ~30 mm/s (typical walking)
Meaning	Speed in the direction the fly is heading

Computation: Projection of the velocity vector onto the heading direction: `fv = vx *`

`cos(theta) + vy * sin(theta)`. Position is Gaussian-smoothed before differentiation. Negative values and values > 50 mm/s are filtered as artifacts.

Interpretation: A stationary fly has fv near 0. Active walking is typically 5–20 mm/s.

8.2 Angular Velocity (av_data)

Property	Value
Units	deg/s
Range	approximately -500 to $+500$ deg/s
Meaning	Rate of heading change (turning speed)

Computation: Least-squares line fit to heading data in a 16-frame sliding window. Positive values indicate counterclockwise turns; negative values indicate clockwise turns.

Interpretation: In the optomotor response, flies turn in the direction of stimulus motion. A positive mean angular velocity during a CW stimulus indicates the fly is following the pattern.

8.3 Turning Rate (curv_data)

Property	Value
Units	deg/mm
Range	Variable
Meaning	Path curvature – how sharply the fly turns per unit distance

Computation: `curv = av_data / fv_data`. Undefined (NaN) when forward velocity < 0.1 mm/s.

Interpretation: High absolute values indicate sharp turns; near-zero values indicate straight walking. This metric normalizes turning by walking speed.

8.4 Distance from Center (dist_data)

Property	Value
Units	mm
Range	0 – 120 mm
Meaning	Distance from the fly to the arena center

Computation: `dist = 120 - d_wall`, where `d_wall` is FlyTracker's distance-to-wall output and 120 mm is the arena radius.

Interpretation: Flies near the center have low dist values; flies near the wall have values approaching 120 mm. Freely-walking flies often exhibit thigmotaxis (preference for walking near the wall).

8.5 Three-Point Velocity (vel_data)

Property	Value
Units	mm/s
Range	0 – ~50 mm/s
Meaning	Overall speed regardless of heading direction

Computation: Central-difference approximation of speed from x,y positions. Always positive (magnitude, not direction).

Interpretation: Unlike forward velocity, this captures all motion including sideways movements. Useful as a general activity measure.

8.6 Viewing Distance (`view_dist`)

Property	Value
Units	mm
Range	0 – ~240 mm (diameter)
Meaning	Distance from fly to arena wall in the heading direction

Computation: Ray-circle intersection from the fly's position along its heading vector to the arena wall.

Interpretation: A fly facing the nearby wall has a short viewing distance; a fly in the center facing across the arena has a large viewing distance. This metric is important for understanding how the visual environment varies with position and orientation.

8.7 Inter-Fly Distance (`IFD_data`)

Property	Value
Units	mm
Meaning	Distance to the nearest other fly

Interpretation: Low values indicate flies are close together; useful for studying social interactions or grouping behavior.

8.8 Inter-Fly Angle (`IFA_data`)

Property	Value
Units	deg
Meaning	Angle to the nearest fly, relative to the focal fly's heading

Interpretation: 0 degrees means the nearest fly is directly ahead; 180 degrees means it is directly behind.

8.9 Position and Heading

- **`x_data`**, **`y_data`**: Position in mm, relative to the arena center
- **`heading_data`**: Unwrapped heading in degrees (continuous, can exceed 360)
- **`heading_wrap`**: Wrapped heading in degrees (-180 to +180)

These are the raw tracking outputs (after filtering and interpolation) that all other metrics are derived from.

9 Visualization and Plotting Functions

At a Glance

- `make_overview` generates summary histograms of locomotor parameters
- `plot_all_features_filt` and `plot_all_features_acclim` plot full-experiment timeseries
- `plot_allcond_onecohort_tuning` creates per-condition comparison plots

9.1 `make_overview.m`

Generates a summary overview figure with histograms of key locomotor parameters for quality checking a single experiment. Called by `process_data_features`.

9.2 `plot_all_features_filt.m`

Plots timeseries of all behavioral features for the full experiment duration (after filtering). Shows forward velocity, angular velocity, turning rate, and distance from center as stacked panels. Called by `process_data_features`.

9.3 `plot_all_features_acclim.m`

Similar to `plot_all_features_filt` but focused on the acclimation periods only. Useful for verifying baseline behavior before stimulus onset.

9.4 `plot_allcond_onecohort_tuning.m`

Creates per-condition comparison plots for a single cohort. Shows stimulus-evoked responses side by side across conditions, making it easy to compare the optomotor response at different speeds or spatial frequencies.

10 Utility Functions

10.1 `check_strain_typos.m`

Corrects common strain name typos that may have been entered through the metadata GUI. Maps misspelled variants to canonical strain names, ensuring consistent naming across the dataset.

10.2 `generate_circ_stim_ufmf.m`

Generates a UFMF video of the LED stimulus pattern (circular projection) for documentation and verification purposes. Creates a video showing what the flies see from above.

11 Potential Pitfalls

When adapting this pipeline to a new experimental setup, protocol, or fly strain, be aware of the following assumptions that are hardcoded in the current codebase. Each can cause silent errors if not addressed.

11.1 Hardcoded Arena Parameters

`calculate_viewing_distance.m` contains hardcoded calibration constants:

```
PPM = 4.1691;          % pixels per mm
CoA = [528, 520]/PPM;  % arena center in mm
R   = 496/PPM;         % arena radius in mm
```

These values are specific to the current camera and arena setup. If the camera, lens, or arena changes, these must be recalibrated. Incorrect values will silently corrupt all viewing distance and distance-from-center metrics.

Source: `processing_functions/functions/calculate_viewing_distance.m`, lines 39–43.

11.2 Hardcoded Project Paths

`process_freely_walking_data.m` sets an absolute project root:

```
PROJECT_ROOT = '\Users\burnettl\...';
```

This path must be updated for any other machine before running the processing pipeline.

Source: `processing_functions/process_freely_walking_data.m`, line 42.

11.3 Protocol Condition Arrays Are Per-Protocol

`comb_data_one_cohort_cond.m` contains a large `if/elseif` block that maps each protocol name to its condition array. New protocols must either:

(a) Store conditions in `LOG.meta.cond_array` (as `protocol_24` and `protocol_27` already do — this is the recommended approach), or

(b) Add a new `elseif` branch to this function.

If neither is done, condition matching will silently fail and produce incorrect results.

Source: `processing_functions/functions/comb_data_one_cohort_cond.m`, lines 3–126.

11.4 Exactly 2 Repetitions Assumed

`comb_data_one_cohort_cond.m` splits the `log_` fields into the first half (Repetition 1) and second half (Repetition 2):

```
if log_n < (n_cond/2) + 1
    rep_str = 'R1_condition_';
else
    rep_str = 'R2_condition_';
end
```

Running 3 or more repetitions, or only 1, will mislabel conditions. The code does not verify that the number of logged conditions equals exactly 2 times the number of unique conditions.

Source: `processing_functions/functions/comb_data_one_cohort_cond.m`, line 203.

11.5 Velocity Threshold of 50 mm/s

`combine_data_one_cohort.m` flags frames where velocity exceeds 50 mm/s as tracking errors and sets them to NaN:

```
vals_to_rm = abs(vel_data(k, :)) > 50;
```

This threshold is appropriate for *Drosophila melanogaster* but may need adjustment for larger or faster species. Real high-speed behaviour is indistinguishable from tracking errors at this threshold.

Source: `processing_functions/functions/combine_data_one_cohort.m`, lines 66 and 130.

11.6 300-Frame Pre-Buffer Is Hardcoded

`comb_data_one_cohort_cond.m` always includes 300 frames (10 seconds at 30 fps) before each condition onset:

```
framesb4 = 300;
start_f = Log.start_f(1) - framesb4;
```

If a stimulus starts fewer than 300 frames after acclimation ends, this can produce out-of-bounds frame indices.

Source: `processing_functions/functions/comb_data_one_cohort_cond.m`, line 228.

11.7 LOG Field Naming Is Fragile

Condition discovery searches for fields whose names start with `log_`:

```
logfields = fields(startsWith(fields, 'log_'));
```

Any extra field added to the LOG struct with a name beginning with `log_` (e.g., `log_notes`, `log_metadata`) will be incorrectly treated as a stimulus condition.

Source: `processing_functions/functions/comb_data_one_cohort_cond.m`, line 195.

11.8 Pattern-to-Description Mapping

`get_title_from_meta.m` uses a large switch statement to map pattern IDs to human-readable descriptions. New patterns require adding a manual entry. Additionally, some patterns have hidden

speed multipliers — patterns with 2-pixel animation steps (e.g., patterns 26, 27, 63) automatically double the nominal speed value, which is easy to overlook.

Source: `processing_functions/functions/get_title_from_meta.m`.

11.9 Tracking QC Uses Statistical Mode

`check_tracking_FlyTrk.m` removes any fly whose frame count differs from the mode (most common frame count):

```
d = [trx.nframes];
nframes_video = mode(d);
```

If a systematic tracking issue causes all flies to be off by one frame, no flies will be flagged. Conversely, if tracking splits are common, valid flies may be removed because their frame count differs from the mode.

Source: `processing_functions/functions/check_tracking_FlyTrk.m`.

11.10 Strain Typos Pass Silently

`check_strain_typos.m` corrects only a predefined list of known typos. New misspellings are not caught and will create separate strain folders in the results directory, splitting data for the same genotype across multiple entries. Always verify strain names after data collection.

Source: `processing_functions/functions/check_strain_typos.m`.

11.11 Move-Folder Nesting Bug in Daily Processing

In `daily_processing.py`, the `move_folder()` function uses `shutil.move(src, dst)`. If the destination folder already exists, `shutil.move` places the source folder *inside* the destination, creating a nested structure (e.g., `2_processed/2025_04_18/2025_04_18/...`). The function includes a warning comment but does not prevent this.

By contrast, `reprocessing_script.py`'s `move_folder()` deletes the destination first with `shutil.rmtree(dst)` before moving – which avoids nesting but is destructive.

Source: `daily_processing/daily_processing.py`, lines 161–183; `daily_processing/reprocessing_script.py`, lines 105–119.

11.12 No Retry Logic for Network File Operations

All automation scripts log network copy/move failures but do not retry them. A transient network outage during `shutil.copytree()` or `shutil.move()` will leave data in an intermediate state (partially copied, or stuck in the source directory). Manual intervention is required to re-run or complete the transfer.

Source: All three main automation scripts (`monitor_and_copy.py`, `monitor_and_track.py`, `daily_processing.py`).

11.13 Duplicated Path Constants Across Automation Scripts

Each automation script defines its own copy of network and local path constants. There is no shared configuration file. If a path changes (e.g., the network share is moved), every script must be updated independently. Currently, `daily_processing.py` and `reprocessing_script.py` contain nearly identical path blocks.

Source: Path constants at the top of `monitor_and_copy.py`, `monitor_and_track.py`, `daily_processing.py`, and `reprocessing_script.py`.

A Function Index

Alphabetical listing of all MATLAB functions referenced in this guide. Page numbers indicate the primary documentation location.

Function	Page	Description
batch_track_ufmf	14	Batch FlyTracker processing for all videos in a folder
calculate_distance_to_nearest_fly	34	Compute IFD and IFA for each fly
calculate_three_point_velocity	32	Central-difference speed estimate
calculate_viewing_distance	33	Ray-circle intersection for viewing distance
check_strain_typos	43	Correct common strain name misspellings
check_tracking_FlyTrk	31	QC: remove bad fly tracks by frame count
comb_data_across_cohorts_cond	35	Merge experiments into hierarchical DATA struct
comb_data_one_cohort_cond	34	Split continuous data by condition using LOG
combine_data_one_cohort	31	Core: compute 12 behavioral metrics
export_to_google_sheets	23	Push metadata to Google Sheet via HTTP
gaussian_conv	32	Gaussian smoothing for position data
generate_circ_stim_ufmf	43	Generate stimulus video for documentation
get_input_parameters	18	GUI dialog for experiment metadata
get_temp_rec	19	Read temperature from DAQ probes
initialize_temp_recording	19	Configure DAQ for temperature monitoring
initialize_video_and_folders	18	Camera setup, folder creation, LOG init
make_overview	42	Summary histograms for QC
plot_all_features_acclim	42	Acclimation period timeseries plots
plot_all_features_filt	42	Full-experiment timeseries plots
plot_allcond_onecohort_tuning	42	Per-condition comparison plots
present_acclim_off	23	Dark acclimation recording
present_fixation_stimulus	22	Static bar display for fixation
present_optomotor_stimulus	21	Standard bidirectional stimulus
present_optomotor_stimulus_curtain	22	Curtain pattern stimulus
present_optomotor_stimulus_curtain_w_interval	22	Curtain with per-trial intervals
present_optomotor_stimulus_diff_contrasts	22	Contrast variation stimulus
present_optomotor_stimulus_one_direction	22	Single-direction stimulus
present_optomotor_stimulus_w_interval	21	Stimulus with per-trial intervals
present_shifted_stimulus	23	Open-loop shifted patterns
process_data_features	30	Single-experiment processing
process_freely_walking_data	29	Top-level processing by date
vel_estimate	32	Sliding-window angular velocity

Automation Scripts (Python)		
monitor_and_copy.py	12	Watchdog-based copy from acquisition PC to network
monitor_and_track.py	13	Polling-based automated FlyTracker batch tracking
daily_processing.py	14	Daily MATLAB processing and result distribution